SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

HIGH PERFORMANCE COMPUTING (BCSE414L)

Review 2: High Performance Computing Course Project

1. Title

Skew-Aware Adaptive Parallel Near-Duplicate Detection on Multicore CPUs

Group Details

Group Number:	29
Submitted By:	Lakshmi Sarupa Venkadesh
Register No.:	23BCE0463
Course Code:	BCSE414L
Faculty Guide:	Vimala Devi K.
Academic Year:	2026–27

Table of Contents

1. Title **1**

Group Details **1**

2. Abstract **3**

3. Introduction **3**

3.1 Background and Motivation 3
3.2 The Pairwise Comparison Bottleneck 3
3.3 The Workload Skew Problem on Multicore CPUs 3
3.4 Project Objectives 3

4. Literature Survey / Existing Work **4**

5. Proposed Methodology with Novelty **5**

5.1 Dataset Generation 5
5.2 Document Fingerprinting 5
5.3 Candidate Generation 5
5.4 Similarity Verification 6
5.5 Sequential Baseline 6
5.6 Standard Parallel Implementation (OpenMP) 6
5.7 Proposed Workload-Aware Adaptive Scheduling 6
5.8 Experimental Evaluation 6
5.9 Novelty in the Proposed Work 7
5.10 Deliverables & Expected Outcomes 7

6. Implementation with Results and Performance Analysis **7**

6.1 Implementation Overview 8
6.2 Experimental Setup 10
6.3 Implementation Results 11
6.4 Performance Analysis and Discussion 19

7. Conclusion **20**

8. References **21**

2. Abstract

Near-duplicate detection identifies documents in a collection that contain the same or nearly the same text. Because checking all document pairs directly scales quadratically ($\mathcal{O}(N^2)$), fingerprinting and band indexing are used to group likely matches into hash buckets before verification. However, uneven duplicate cluster sizes create severe quadratic skew in per-bucket comparison work ($\frac{k(k-1)}{2}$), causing load imbalance and candidate-merging bottlenecks when parallelized across CPU cores.

This report presents the design, C++17/OpenMP implementation, and experimental evaluation of a parallel near-duplicate detection system using 64-bit SimHash fingerprinting, 8-band candidate generation, and Hamming-distance verification ($d_H \leq 3$). A single-threaded sequential baseline and three fixed OpenMP loop schedules (`Static`, `Dynamic`, and `Guided`) were implemented and compared against a workload-aware `Adaptive` pipeline that selects ordinary-bucket scheduling using the bucket work Coefficient of Variation (CV), splits oversized buckets ($W_b > 4.0\overline{W}$) into 4,096-pair chunks via triangular index unranking, merges and deduplicates thread-local candidates in parallel across ID-range partitions, and optionally filters repeated multi-band pairs at emission time using a branch-free bitwise SWAR zero-byte test (`--band-suppression`).

Benchmarks across synthetic `Balanced` and power-law `Skewed` datasets ($N \in \{500, 1000, 2000, 4000, 8000\}$) on a 4-core / 8-thread Intel Core i5-1135G7 CPU (3,400 measured runs plus 680 warm-up runs across Phases 8–10) confirmed 100% exact output parity with the sequential baseline. On $N = 8,000$ `Skewed` (CV = 13.55, where 17 buckets hold 90.16% of all 8.62M pair evaluations), `Adaptive` scheduling kept thread work imbalance down to $I = 1.078$ at $T = 4$, parallel candidate aggregation reduced Stage 3e merge/sort/dedup time by 63.2% (465.34 ms → 171.11 ms at $T = 8$), and enabling SWAR band suppression eliminated 68.35% (5.89M) of redundant candidate emissions, cutting `Adaptive` ($T = 8$) end-to-end runtime by another 49.6% (294.95 ms → 148.72 ms) for a 4.41× total speedup over the baseline sequential pipeline.

3. Introduction

3.1 Background and Motivation

Large document collections, such as web pages, digital libraries, legal archives, and medical records, often contain many duplicate or near-duplicate files caused by copying, shared templates, and minor edits [1], [4], [5]. Finding these near-duplicates quickly is important for saving storage space, improving search results, and cleaning datasets before analysis.

3.2 The Pairwise Comparison Bottleneck

For a collection of N documents, checking every possible pair requires $\frac{N(N-1)}{2} = \mathcal{O}(N^2)$ comparisons. As N gets larger, comparing all pairs directly becomes too slow. Locality-Sensitive Hashing (LSH) methods such as MinHash [1] and Charikar's SimHash [2] solve this by turning each document into a short binary fingerprint where similar documents have similar fingerprints. By splitting each fingerprint into bands, only documents that match in at least one band need to be compared as candidate pairs [4].

3.3 The Workload Skew Problem on Multicore CPUs

Even though banding filters out most unrelated pairs, real document collections rarely spread documents evenly across hash buckets [3], [6], [7]. Common templates or frequently copied texts fall into the same buckets. Since a bucket with k documents produces $\frac{k(k-1)}{2}$ pairs to check, even a moderate increase in bucket size causes a huge jump in comparison work. When standard OpenMP schedules (`static`, `dynamic`, or `guided`) assign whole buckets to threads, one thread can get stuck working on a giant bucket while other cores finish early and wait, or dynamic scheduling can add unnecessary overhead on tiny buckets.

3.4 Project Objectives

The main goals of this project are to:

1. Build a synthetic dataset generator that can create both balanced and skewed duplicate groups.
2. Implement a sequential baseline pipeline using SimHash fingerprinting, band-based candidate generation, and Hamming-distance verification.
3. Parallelize the pipeline in C++17 using OpenMP and test standard scheduling modes (`static`, `dynamic`, and `guided`).
4. Design and implement a **workload-aware adaptive scheduling method** that estimates the work in each bucket before execution and breaks very large buckets into smaller parallel tasks.
5. Measure execution time, speedup, parallel efficiency, and load balance across different dataset sizes, skew levels, and thread counts.

4. Literature Survey / Existing Work

1. On the Resemblance and Containment of Documents

> ***Broder** [1] introduced the mathematical model for measuring document similarity and containment using MinHash and random sampling. This paper showed how similar documents can be found without comparing every pair of documents directly.*

2. Similarity Estimation Techniques from Rounding Algorithms

> ***Charikar** [2] introduced locality-sensitive hashing based on random hyperplane rounding, known as **SimHash**. This work showed that the similarity between two high-dimensional feature vectors can be estimated from the Hamming distance between their compact binary fingerprints.*

3. LSH Forest: Self-Tuning Indexes for Similarity Search

> ***Bawa, Condie, and Ganesan** [3] proposed the LSH Forest index to handle different similarity thresholds without manual tuning, and showed how uneven data distributions can slow down similarity searches.*

4. Probabilistic Near-Duplicate Detection Using SimHash

> ***Sood and Loguinov** [4] analyzed how SimHash performs on large web datasets. They showed that while SimHash is very good at filtering out unrelated documents, generating and checking the remaining candidate pairs still takes significant compute time.*

5. Near-Duplicate Detection in an Academic Digital Library

> ***Williams and Giles** [5] tested SimHash and shingle-based duplicate detection on the CiteSeerX digital library, comparing fingerprint size, filtering speed, and detection accuracy.*

6. A Scalable Parallel Deduplication Algorithm

> ***Santos et al.** [6] built parallel deduplication algorithms on multicore CPUs and showed that uneven candidate cluster sizes cause threads to become unbalanced unless the workload is split carefully.*

7. Efficient String Similarity Join in Multi-Core and Distributed Systems

> ***Yan et al.** [7] studied parallel string similarity joins on multicore and distributed systems, showing that splitting data based on item frequency helps avoid bottlenecks from skewed inputs.*

8. Parallel Loop Scheduling for High Performance Computers

> ***Yue and Lilja [8]*** *compared static, dynamic, and guided loop scheduling in parallel computing and showed that no single fixed schedule works best for every type of workload.*

9. Accelerating Content-Defined-Chunking-Based Data Deduplication by Exploiting Parallelism

> ***Xia et al. [9]*** *developed P-Dedupe, a parallel deduplication system that separates fingerprinting and verification into parallel stages with dynamic task assignment to keep multicore CPUs busy.*

10. Parallel Near-Duplicate Document Detection Using General-Purpose GPU

> ***Peshevski, Zdraveski, and Ristov [10]*** *built a GPU-based MinHash pipeline for near-duplicate document detection, showing that fine-grained parallelism works well for fingerprint-based similarity search.*

11. Automatic Near-Duplicate Content Document Detection in a Cancer Registry

> ***Niemi et al. [11]*** *used SimHash fingerprinting followed by sequence alignment to find near-duplicate medical reports in a cancer registry, showing that fast fingerprint filtering paired with exact verification works well on real text data.*

5. Proposed Methodology with Novelty

The system is implemented in C++17 using OpenMP for shared-memory parallelism.

```
Input Documents —> SimHash Fingerprinting —> Band Partitioning & Candidate Generation —> Skew-Aware
                    Adaptive Parallel Verification (OpenMP) —> Final Duplicate Pairs
```

5.1 Dataset Generation

A synthetic dataset generator is built to create original documents, exact copies, slightly modified copies, and unrelated documents. The generator supports configurable document lengths and controls how evenly or unevenly duplicates are grouped (balanced vs. skewed distributions). A fixed random seed is used so all experiments can be repeated on the exact same data.

5.2 Document Fingerprinting

Each document is broken into small word groups (shingles) and hashed into a 64-bit SimHash fingerprint. Documents with similar wording produce fingerprints that differ in only a few bit positions.

5.3 Candidate Generation

To avoid checking all $O(N^2)$ document pairs, each 64-bit fingerprint is split into fixed-size bands. Documents that have the same band value in the same band position are placed into the same hash bucket and treated as candidate pairs. Since the same pair of documents can match in more than one band, duplicate candidate pairs are removed before the final check.

The candidate generation step is also checked against a brute-force all-pairs comparison on small test datasets to make sure near-duplicate pairs are not missed.

5.4 Similarity Verification

For each candidate pair, the system calculates the Hamming distance between their two 64-bit fingerprints using a bitwise XOR operation and bit counting. Any pair whose Hamming distance is less than or equal to a threshold k is reported as a near-duplicate.

5.5 Sequential Baseline

A single-threaded sequential version of the pipeline is implemented first. It is used both to check that the parallel results are correct and as the baseline time (T_1) for calculating speedup.

5.6 Standard Parallel Implementation (OpenMP)

OpenMP is used to run the pipeline across multiple CPU threads. Three built-in loop scheduling policies are tested:

- **Static Scheduling**: Divides loop iterations into fixed chunks ahead of time. It has very low overhead, but threads can become unbalanced if some buckets are much larger than others.
- **Dynamic Scheduling**: Assigns chunks to threads as they become free. This helps balance uneven work, but adds synchronization overhead.
- **Guided Scheduling**: Starts with larger chunks and gradually uses smaller chunks to balance overhead and load distribution.

All scheduling modes run on the exact same input documents and candidate logic so that any runtime differences come only from how the parallel work is scheduled.

5.7 Proposed Workload-Aware Adaptive Scheduling

For a bucket with k documents, the number of pairs to process grows quadratically:

$$\text{Cost}(k) = \frac{k(k-1)}{2}$$

When bucket sizes are skewed, a few very large buckets take up most of the execution time. The proposed adaptive approach works in four steps:

1. **Work Estimation**: Calculate the number of pairs $\frac{k(k-1)}{2}$ in each bucket before starting the parallel comparison phase.
2. **Bucket Classification**: Separate normal-sized buckets from oversized buckets based on the distribution of bucket work.
3. **Splitting Large Buckets**: Break oversized buckets into smaller sub-tasks of pairs instead of giving an entire huge bucket to a single thread.
4. **Parallel Execution**: Schedule the normal buckets and the split sub-tasks across OpenMP threads so all cores stay evenly loaded.

This adaptive approach is tested directly against the sequential baseline and the standard OpenMP schedules.

5.8 Experimental Evaluation

The experiments test different settings for:

- **Dataset Size (N)**: Small to larger document collections.
- **Workload Skew**: Balanced datasets vs. skewed datasets with heavy duplicate clusters.
- **Thread Count**: 1, 2, 4, 8, and the maximum available CPU threads.

- **Scheduling Policies**: Sequential, OpenMP Static, OpenMP Dynamic, OpenMP Guided, and Proposed Adaptive.
- **Chunk Sizes**: Different chunk size settings for scheduling.

Performance Metrics:

- **Execution Time (T_p)**: Runtime measured across repeated runs.
- **Speedup (S_p)**: $S_p = \frac{T_1}{T_p}$
- **Parallel Efficiency (E_p)**: $E_p = \frac{S_p}{p}$
- **Load Imbalance**: Ratio of maximum per-thread pair work to mean per-thread pair work.
- **Candidate Buffer Storage**: Estimated memory required for raw and deduplicated candidate-pair buffers based on pair counts.
- **Correctness & Recall Checks**: Checking candidate-generation recall against brute-force all-pairs comparison on small test datasets and verifying exact output parity between the parallel and sequential pipelines.

5.9 Novelty in the Proposed Work

Prior studies in the literature survey cover document fingerprinting [1], [2], [4], [5], similarity indexing under data skew [3], [6], [7], and loop scheduling [8] mostly as separate problems. When parallelizing band-based candidate generation with standard OpenMP loop schedules, a straightforward bucket-level loop assigns each hash bucket as a single iteration, which can struggle when bucket sizes vary widely:

1. **Using Known Bucket Cost Before Scheduling**: Standard OpenMP loop schedules do not inspect the $\frac{k(k-1)}{2}$ pair cost inside each bucket of size k. The proposed approach calculates this exact pair count ahead of time.
2. **Handling Heavy Skew on Multicore CPUs**: In near-duplicate detection, a few buckets can contain thousands of pairs while most buckets have very few. Splitting only the heavy buckets prevents a single slow thread from holding up the rest of the CPU cores.
3. **Combining Bucket Statistics with OpenMP Scheduling**: The scheduler looks at how uneven the bucket work is at runtime to decide how to schedule normal buckets while dynamically splitting heavy buckets.
4. **Testing When Adaptive Scheduling Helps Most**: Experiments are run across both balanced and skewed datasets to see clearly when adaptive scheduling beats standard OpenMP policies and when simple static or guided scheduling is already enough.

5.10 Deliverables & Expected Outcomes

- **Course Deliverable**: A complete project report in **IEEE conference format** covering the design, implementation, hardware setup, speedup, efficiency, and load-balance results.
- **Codebase**: A modular C++17 and OpenMP codebase with a synthetic dataset generator, SimHash fingerprinting, standard and adaptive parallel schedulers, and benchmark scripts.
- **Expected Outcome**: Clear benchmark results showing how workload skew affects parallel near-duplicate detection and how much adaptive bucket splitting improves load balance and runtime.

6. Implementation with Results and Performance Analysis

Following the Review 1 proposal, the complete system was built, tested, and benchmarked across development phases (Phases 0 through 10). This section covers the C++17/OpenMP implementation, the benchmark

setup, the experimental results across all scheduling modes and optimizations, and an analysis of what the results show.

6.1 Implementation Overview

Development Environment and Code Structure

The project was implemented in **C++17** using **OpenMP** (`libgomp`) for shared-memory parallelism and compiled with 64-bit **GCC 16.2.0** (MSYS2 UCRT64) using **CMake 4.4.4** and **Ninja**. The codebase is split into a core static library (`libhpc_core.a`), two executables (`hpc_near_duplicate.exe` for demo runs and `hpc_benchmark.exe` for automated benchmarking), and six unit-test executables run through **CTest**:

- `include/document.hpp`: Defines `Document` (`id`, `group_id`, `kind`, `source_id`, `text`) and `DocumentKind` (`Original`, `ExactCopy`, `ModifiedCopy`, `Unrelated`).
- `include/dataset_generator.hpp` & `src/dataset_generator.cpp`: Implements deterministic synthetic dataset generation (`generate_dataset`) supporting `Balanced` (round-robin) and `Skewed` ($w_i = 1/(i+1)^2$ power-law) duplicate group distributions using `std::mt19937_64`.
- `include/simhash.hpp` & `src/simhash.cpp`: Implements word tokenization, overlapping 3-word shingle generation (`kDefaultShingleSize = 3`), 64-bit FNV-1a hashing (`fnv1a_hash64`), 64-bit SimHash bit voting (`compute_simhash`), and bitwise XOR + hardware popcount Hamming distance (`hamming_distance`).
- `include/candidate_gen.hpp` & `src/candidate_gen.cpp`: Implements the sequential baseline (`run_sequential_pipeline`), 8-band × 8-bit bucket partitioning (`2,048` buckets total), bucket workload statistics (`BucketWorkloadStats`), reference and SWAR earlier-band duplicate checks (`matched_in_earlier_band_ref`, `matched_in_earlier_band_swar`), and Hamming verification.
- `include/parallel_pipeline.hpp` & `src/parallel_pipeline.cpp`: Implements fixed OpenMP scheduling baselines (`run_parallel_pipeline` supporting `Static`, `Dynamic`, and `Guided` via an RAII `OmpScheduleGuard`), the workload-aware adaptive pipeline (`run_adaptive_pipeline`), heavy-bucket chunk splitting (`split_bucket_into_chunks`) and pair index unranking (`unrank_pair_position`), and parallel range-partitioned candidate merging (`aggregate_and_deduplicate_candidates`).
- `include/benchmark.hpp`, `src/benchmark.cpp`, & `src/benchmark_main.cpp`: Implements the benchmark runner (`run_benchmark_suite`), randomized per-round execution ordering (`generate_shuffled_config_order`), paired in-process Baseline-vs.-Suppression comparisons (`generate_paired_config_order`), output parity checks (`verify_pipeline_parity`), and CSV output.

Pipeline Stages and Scheduling Implementations

Every pipeline mode (`Sequential`, `Static`, `Dynamic`, `Guided`, and `Adaptive`) runs on the input `std::vector<Document>` in four separately timed stages:

1. **Stage 1: Document Fingerprinting (`fingerprinting_ms`):** After checking document IDs and building an ID-to-index lookup table, each document is hashed into a 64-bit SimHash fingerprint. The parallel pipelines split documents across threads with `#pragma omp parallel for schedule(static)`, writing directly to `fingerprints[i]` without needing locks.
2. **Stage 2: Bucket Construction & Workload Statistics (`bucket_construction_ms`):** Each 64-bit fingerprint is split into 8 disjoint 8-bit bands (`band_index` $\in [0,7]$, `band_value` $\in [0,255]$) and placed into a flat array of $2,048$ buckets. By the Pigeonhole Principle, any two fingerprints with Hamming distance ≤ 7 must share at least one identical 8-bit band, which guarantees 100% algorithmic candidate recall for the verification threshold $k = 3$. Since filling $2,048$ vectors takes only 1.0–2.5 ms even at $N = 8,000$, this step runs on a single thread to avoid lock contention, and then computes `occupied_buckets` ($k_b \geq 1$), `positive_work_buckets` ($k_b \geq 2$), total pair work $\sum W_b$, mean bucket work $\overline{W}$, and standard deviation σ (CV $= \sigma/\overline{W}$).
3. **Stage 3: Candidate Generation, Merging, and Deduplication (`candidate_gen_total_ms`):**

- **Fixed OpenMP Baselines (`run_parallel_pipeline`):** Loop over all positive-work buckets using `#pragma omp parallel for schedule(runtime)` set to `Static`, `Dynamic`, or `Guided`. Each thread writes canonical pairs $(\min(u,v), \max(u,v))$ into its own thread-local buffer `per_thread_candidates[tid]`.
- **Workload-Aware Adaptive Pipeline (`run_adaptive_pipeline`):** Breaks Stage 3 into five timed sub-stages:
  - *Stage 3a (`adaptive_decision_ms`):* Runs `select_ordinary_schedule_policy()` (`Static` if CV ≤ 1.0, `Dynamic` if CV > 1.0) and separates normal buckets ($W_b \leq 4.0\overline{W}$) from oversized buckets ($W_b > 4.0\overline{W}$).
  - *Stage 3b (`ordinary_candidate_gen_ms`):* Processes normal buckets in parallel using the chosen policy (`Static` or `Dynamic`).
  - *Stage 3c (`heavy_chunk_gen_ms`):* Splits each oversized bucket into `HeavyBucketChunk` tasks of at most `max_pair_task_size = 4,096` pairs.
  - *Stage 3d (`heavy_pair_proc_ms`):* Runs all heavy-bucket chunks in parallel using `#pragma omp parallel for schedule(dynamic, 1)`. At the start of each chunk, `unrank_pair_position(k, start_pair_pos)` uses integer binary search over row_start$(i,k) = \frac{i(2k-1-i)}{2}$ to find the starting pair (i,j) in $\mathcal{O}(\log k)$ time, and then steps through the remaining pairs in $\mathcal{O}(1)$ time.
  - *Stage 3e (`merge_sort_dedup_ms`):* Merges, sorts, and deduplicates all thread-local candidate buffers in parallel using `aggregate_and_deduplicate_candidates()`. This function slices uneven thread buffers into 65,536-pair blocks, scatters pairs lock-free into $P = \min(64, \max(2, 8T))$ ordered ID-range partitions, sorts and deduplicates each partition in parallel (using Longest-Processing-Time-First order, plus a multi-thread sub-chunk sort and merge fallback if one partition holds $> 50\%$ of all pairs), and copies the unique pairs into the final output vector.
- **Optional Band-Aware Candidate Suppression (`--band-suppression`):** Because identical or very similar documents often match in several bands at once, the baseline pipeline emits the same pair multiple times across bands $0 \ldots 7$ and removes the duplicates later during Stage 3e sorting. When `enable_band_suppression = true`, candidate loops in bands $b \in [1,7]$ call `matched_in_earlier_band_swar(fp_u, fp_v, b)` before adding (u,v) to the buffer. This bitwise SWAR check masks `fp_u ^ fp_v` to the lower b bytes and checks if any earlier byte is `0x00`:

  $$\text{has_zero_byte}(x) = (x - 0\text{x}0101010101010101) \;\&\; \sim x \;\&\; 0\text{x}8080808080808080$$

  If an earlier byte is zero, the pair already matched in an earlier band $b' < b$ and is skipped right away, while counters track `raw_candidates_evaluated`, `raw_candidates_emitted`, and `suppressed_pair_count`.

4. **Stage 4: Hamming-Distance Verification (`verification_ms`):** Splits the deduplicated candidate pairs across threads (`schedule(static)` in the adaptive pipeline and `schedule(runtime)` in the fixed parallel pipeline), checks `hamming_distance(fp_u, fp_v) <= 3` using hardware `POPCNT`, and combines the verified pairs in thread order.

Testing, Recall Validation, and Scope of Metrics

Correctness was verified using six CTest suites (`DatasetGeneratorTest`, `SimHashTest`, `CandidateGenTest`, `ParallelPipelineTest`, `AdaptivePipelineTest`, and `BenchmarkTest`) in both `Debug` and `Release` builds:

- **Algorithmic Candidate Recall (100% for $d_H \leq 7$):** `CandidateGenTest` (`test_pigeonhole_candidate_recall`) compares 8-band candidate generation against an exhaustive $\mathcal{O}(N^2)$ all-pairs check and confirms that 100% of document pairs with Hamming distance ≤ 7 are captured as candidates.
- **SWAR Suppression Verification:** `AdaptivePipelineTest` tests `matched_in_earlier_band_swar` against a byte-by-byte reference (`matched_in_earlier_band_ref`) across all band indices $0 \ldots 7$, byte-boundary edge cases (`0x00`, `0x01`, `0x7F`, `0x80`, `0xFF`), and 200,000 random 64-bit XOR values per band.
- **Exact Bitwise Output Parity (100%):** Every benchmark run calls `verify_pipeline_parity()`, confirming that all parallel and adaptive runs (with and without band suppression) produce the exact same

`fingerprints`, sorted `candidate_pairs`, `verified_pairs`, and `workload_stats` as the sequential baseline, and that `raw_candidates_evaluated == raw_candidates_emitted + suppressed_pair_count` always holds.

- **Note on Memory and Accuracy Metrics:** Operating-system resident set size (RSS) and document-level precision/recall against synthetic `group_id` labels were not directly instrumented as benchmark output columns. Instead, memory savings are evaluated via the **calculated `CandidatePair` buffer footprint** (pair count × 16 bytes per `CandidatePair`, shown in **Table 1b**), and detection correctness is enforced via 100% candidate recall ($d_H \leq 7$) and 100% bitwise parity of verified pairs ($d_H \leq 3$) against the sequential reference pipeline.

6.2 Experimental Setup

Hardware and Software Environment

All benchmarks were run on a single multicore laptop:

- **Processor:** Intel Core i5-1135G7 (11th Gen Tiger Lake, 4 physical cores, 8 logical processors with Hyper-Threading, 2.40 GHz base clock).
- **Memory & OS:** 8 GB RAM running 64-bit Windows 11.
- **Compiler & Flags:** 64-bit GCC 16.2.0 (`C:\msys64\ucrt64\bin\g++.exe`), C++17 (`-std=c++17`), OpenMP (`-fopenmp`), compiled in `Release` mode (`-O3 -DNDEBUG`).
- **Timer:** `std::chrono::steady_clock` measuring stage and total wall-clock times in milliseconds (`ms`).

Synthetic Dataset Settings

All experiments used **synthetic datasets** generated in memory before timing started; no external text dataset was used. In the benchmark suite (`BenchmarkSuiteConfig`), datasets were generated with:

- **Dataset Sizes (N):** $N \in \{500, 1000, 2000, 4000, 8000\}$ documents.
- **Document Mix:** Word length between 30 and 60 words (`min_words = 30`, `max_words = 60`), made up of 40% `Original` documents, 25% `ExactCopy` duplicates (`exact_duplicate_ratio = 0.25`), 25% `ModifiedCopy` documents (`modified_copy_ratio = 0.25`, with 10% of words replaced), and 10% `Unrelated` documents (`unrelated_ratio = 0.10`), using `random_seed = 42`.
- **Skew Profiles:** `Balanced` (copies spread evenly round-robin across original groups) and `Skewed` (copies assigned using a power-law weight $w_i = 1/(i+1)^2$ so a few groups get most of the duplicates).
- **SimHash & Adaptive Parameters:** `shingle_size = 3`, `max_hamming_threshold = 3`, `cv_threshold = 1.0`, `oversized_bucket_multiplier = 4.0`, and `max_pair_task_size = 4,096`.

Benchmark Protocol and Run Counts

Each `(dataset_size, profile, seed)` dataset is generated **once** before timing and passed as a `const` reference to all tested configurations (`Sequential` at $T = 1$, and `Static`, `Dynamic`, `Guided`, and `Adaptive` at $T \in \{1, 2, 4, 8\}$, giving 17 configurations per dataset). Each configuration performs **1 unrecorded warm-up run** followed by **5 measured runs**.

To avoid unfair bias from CPU clock speed changes (turbo boost / thermal throttling) or warm caches across back-to-back runs:

1. **Single-Mode Randomized Suites (Phase 8 `phase8_full` & Phase 9 `phase9_agg`):** In each repetition round, the 17 configurations per dataset (10 datasets ×17 = 170 summary rows) are shuffled using `generate_shuffled_config_order` (`order_seed = 1337`). Each suite records 850 **measured runs** in its raw CSV (`170` configurations ×5 repetitions) plus 170 **warm-up runs** (1,020 total pipeline executions per suite).
2. **Paired In-Process Comparison Suite (Phase 10 `phase10_full_paired`):** Running baseline and

band-suppression in separate OS processes during pilot testing caused a 45–58% difference in Stage 1 (`fingerprinting_ms`) from inter-process CPU turbo-boost drift. To fix this, `generate_paired_config_order` runs Baseline (`enable_band_suppression = false`) and Suppression (`enable_band_suppression = true`) back-to-back inside the same process on the same dataset (340 summary rows), alternating which variant runs first across rounds (`((round_index + k) & 1) == 0`). The Phase 10 full paired suite records 1,700 **measured runs** in `results/phase10_full_paired_raw.csv` (340×5) plus 340 **warm-up runs** (2,040 total executions).

3. **Combined Phase 8–10 Run Total:** Across the three full benchmark suites (`phase8_full`, `phase9_agg`, and `phase10_full_paired`), the evaluation comprises 3,400 **measured runs** ($850 + 850 + 1,700$, summarized in 680 summary CSV rows) and 680 **warm-up runs** ($170 + 170 + 340$), for 4,080 **total pipeline executions**.

6.3 Implementation Results

Summary of Main Findings:

- **Extreme Skew in Candidate Generation:** At $N = 8,000$, the `Skewed` profile generates 8.17× more bucket pair evaluations than `Balanced` (8.62M vs. 1.05M), with 90.16% of all pair work concentrated in just 17 heavy buckets (CV = 13.55).
- **Parallel Candidate Aggregation (Phase 9):** Parallelizing Stage 3e (`aggregate_and_deduplicate_candidates`) removed a 79.2% serial bottleneck, cutting Stage 3e time at $N = 8,000$ `Skewed` ($T = 8$) by 63.2% (465.34 ms → 171.11 ms) and nearly doubling `Adaptive` speedup from 1.09× to 2.08×.
- **Band-Aware Duplicate Suppression (Phase 10):** Filtering earlier-band collisions in registers via `matched_in_earlier_band_swar` eliminates 68.35% **(5.89M)** of raw candidate writes on $N = 8,000$ `Skewed`, reducing candidate buffer storage from 137.85 **MB to** 43.63 **MB** and cutting `Adaptive` ($T = 8$) runtime by another 49.6% (294.95 ms → 148.72 ms, 4.41× **total speedup**).
- **Load Balance:** `Adaptive` heavy-bucket splitting keeps thread work imbalance low ($I = 1.078$ at $T = 4$ on $N = 8,000$ `Skewed`) and uses all 8/8 threads on every dataset.

(Note: In Tables 2 and 4 below, negative percentage changes in runtime mean the optimized version is faster, while positive percentages mean it took longer.)

1. Bucket Workload Skew, Candidate Multiplicity, and Estimated Buffer Storage

To keep the metrics readable, **Table 1a** reports the SimHash bucket distribution and adaptive scheduler diagnostics across all ten datasets (`N = 500` to `8,000`), and **Table 1b** reports the corresponding pair evaluation counts, suppressed cross-band duplicates, calculated `CandidatePair` buffer storage (pairs × 16 B), and verified near-duplicate pair counts ($d_H \leq 3$).

Table 1a: Observed SimHash Bucket Workload Skew and Adaptive Scheduler Diagnostics (`seed = 42`)

N	Profile	Occupied Buckets	Pos.-Work Buckets	Max Size (k_{max})	Max Bucket Work (W_{max})	Mean Work ($\overline{W}$)	Bucket CV	Adaptive Ord. Policy	Oversized Buckets	Heavy Chunks
500	Balanced	1,565	1,121	10	45	4.31	1.116	`Dynamic`	26	26
500	Skewed	1,532	845	92	4,186	36.25	8.193	`Dynamic`	18	19
1000	Balanced	1,909	1,703	16	120	10.60	1.122	`Dynamic`	46	46
1000	Skewed	1,890	1,507	212	22,366	97.72	12.012	`Dynamic`	17	43
2000	Balanced	2,037	2,002	30	435	34.35	0.966	`Static`	25	25
2000	Skewed	2,030	1,954	393	77,028	275.97	13.190	`Dynamic`	22	129
4000	Balanced	2,048	2,048	38	703	129.75	0.728	`Static`	9	9
4000	Skewed	2,048	2,044	790	311,655	1,096.34	13.836	`Dynamic`	22	512
8000	Balanced	2,048	2,048	73	2,628	514.63	0.598	`Static`	4	4
8000	Skewed	2,048	2,048	1,534	1,175,811	4,206.91	13.551	`Dynamic`	17	1,904

Table 1b: Candidate Multiplicity, Band Suppression Counts, Calculated Buffer Storage (16 B/pair), and Verified Pairs

N	Profile	Evaluated Pairs ($\sum W_i$)	Unique Candidates	Suppressed Pairs (Supp=1)	Supp. Share (%)	Multiplicity (Eval/Unique)	Raw Buffer Est. (MB)	Supp. Buffer Est. (MB)	Verified Pairs ($d_H \leq 3$)*
500	Balanced	4,836	3,863	973	20.12%	1.25x	0.08	0.06	125
500	Skewed	30,633	9,895	20,738	67.70%	3.10x	0.49	0.16	2,755
1000	Balanced	18,054	15,980	2,074	11.49%	1.13x	0.29	0.26	250
1000	Skewed	147,265	44,784	102,481	69.59%	3.29x	2.36	0.72	13,952
2000	Balanced	68,764	64,145	4,619	6.72%	1.07x	1.10	1.03	500
2000	Skewed	539,236	174,378	364,858	67.66%	3.09x	8.63	2.79	47,913
4000	Balanced	265,730	254,740	10,990	4.14%	1.04x	4.25	4.08	1,000
4000	Skewed	2,240,923	685,559	1,555,364	69.41%	3.27x	35.85	10.97	209,085
8000	Balanced	1,053,961	1,024,755	29,206	2.77%	1.03x	16.86	16.40	2,000
8000	Skewed	8,615,757	2,727,152	5,888,605	68.35%	3.16x	**137.85**	**43.63**	791,879

**Note: `Verified Pairs` is the exact count of candidate pairs with SimHash Hamming distance $d_H \leq 3$ (verified to be 100% identical across all sequential and parallel pipelines), not a document-level precision/recall ratio.*

What Tables 1a and 1b show: On `Balanced` datasets, bucket sizes stay small ($k_{\max} = 73$, $W_{\max} = 2,628$ at $N = 8,000$), CV drops below 1.0 for $N \geq 2,000$ (CV = 0.598 at $N = 8,000$, so the adaptive scheduler selects `Static` for normal buckets), and only 2.77% of evaluated pairs at $N = 8,000$ are multi-band duplicates. On `Skewed` datasets at $N = 8,000$, CV reaches 13.551, the largest bucket holds $k_{\max} = 1,534$ documents ($W_{\max} = 1,175,811$ pairs), and 17 heavy buckets contain $7,767,834$ of the $8,615,757$ total pair evaluations (90.16%). Because 68.35% ($5,888,605$) of those raw pairs are repeat matches from earlier bands (3.16× multiplicity), suppressing them at emission time reduces the calculated raw candidate buffer footprint from 137.85 MB to 43.63 MB (saving 94.22 MB of vector writes).

2. Impact of Parallel Candidate Merging and Deduplication (Pre-Aggregation Baseline vs. Phase 9)

In the initial pre-aggregation benchmarks (`results/benchmark_summary.csv` and Phase 8 `results/phase8_full_summary.csv`), candidate generation ran in parallel across threads, but Stage 3e merged, sorted (`std::sort`), and deduplicated (`std::unique`) all thread buffers on a single thread. **Figures 1–4** (generated from `results/benchmark_summary.csv` via `scripts/plot_benchmarks.py`) show this initial scaling behavior before Stage 3e was parallelized: `Adaptive` achieved the lowest pair-work imbalance ($I = 1.03$–1.08 at $T = 2, 4$), but end-to-end speedup on $N = 8,000$ `Skewed` stalled at 1.09×–1.25× because sequential Stage 3e dominated total runtime.

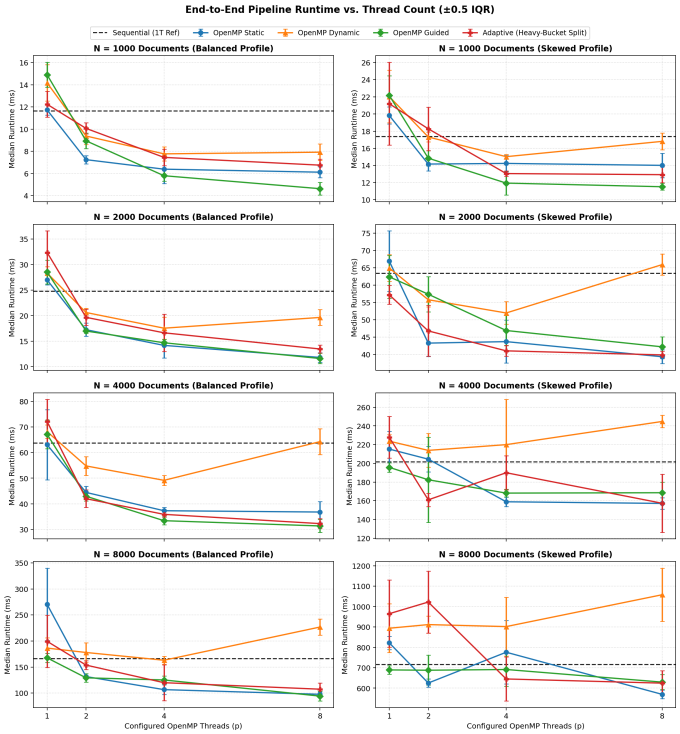


Figure 1: Initial Baseline End-to-End Runtime vs Thread Count (Pre-Aggregation Parallelization)

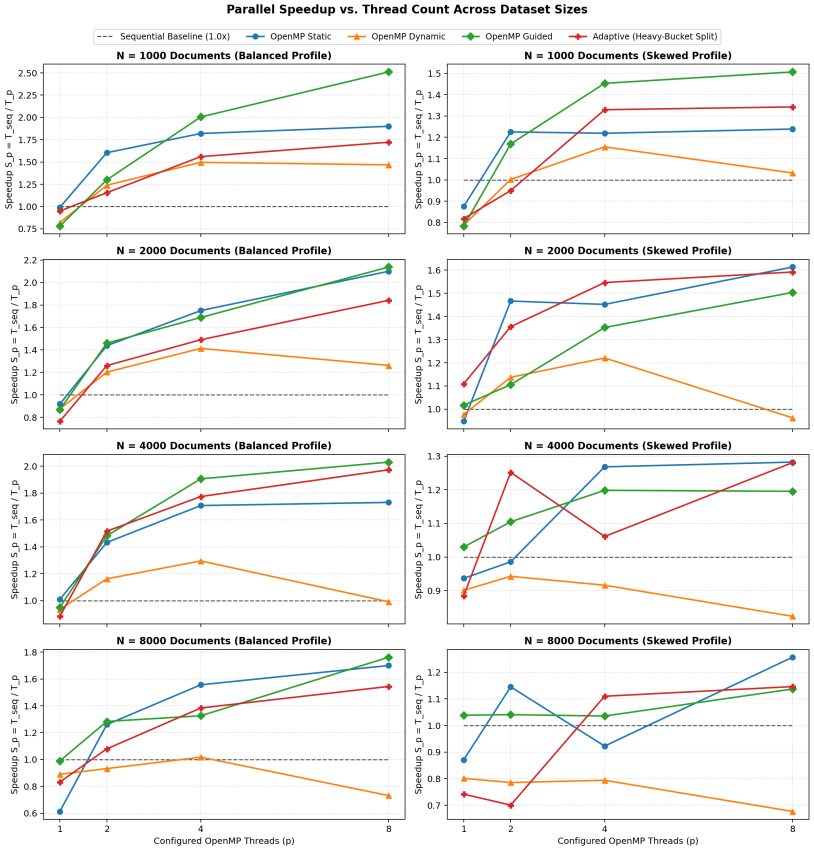


Figure 2: Initial Baseline Parallel Speedup vs Thread Count (Pre-Aggregation Parallelization)

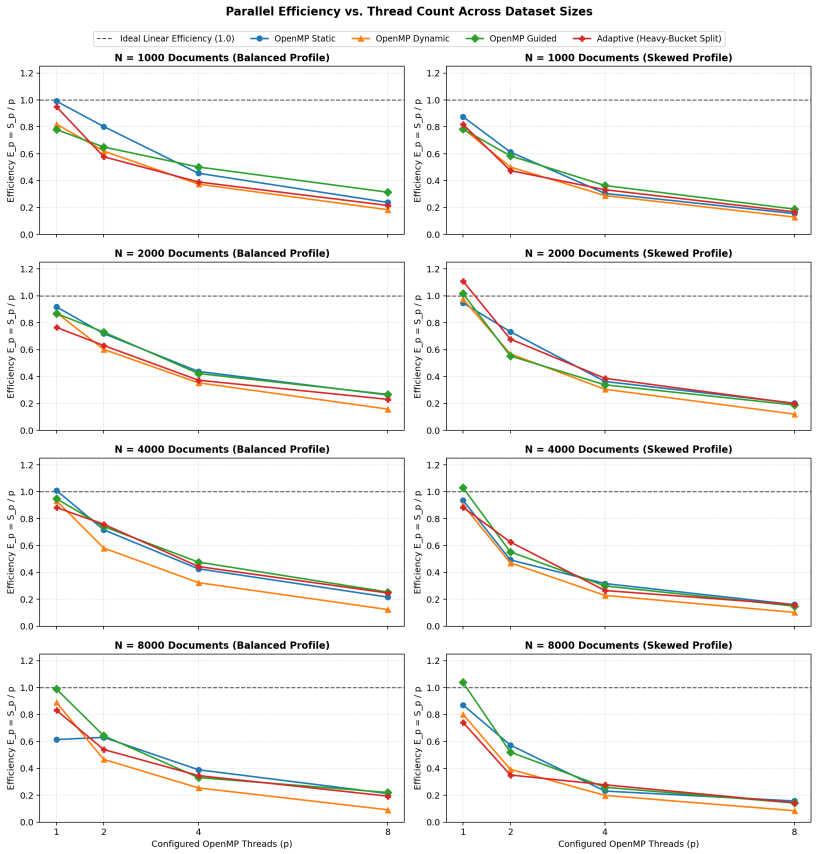


Figure 3: Initial Baseline Parallel Efficiency vs Thread Count (Pre-Aggregation Parallelization)

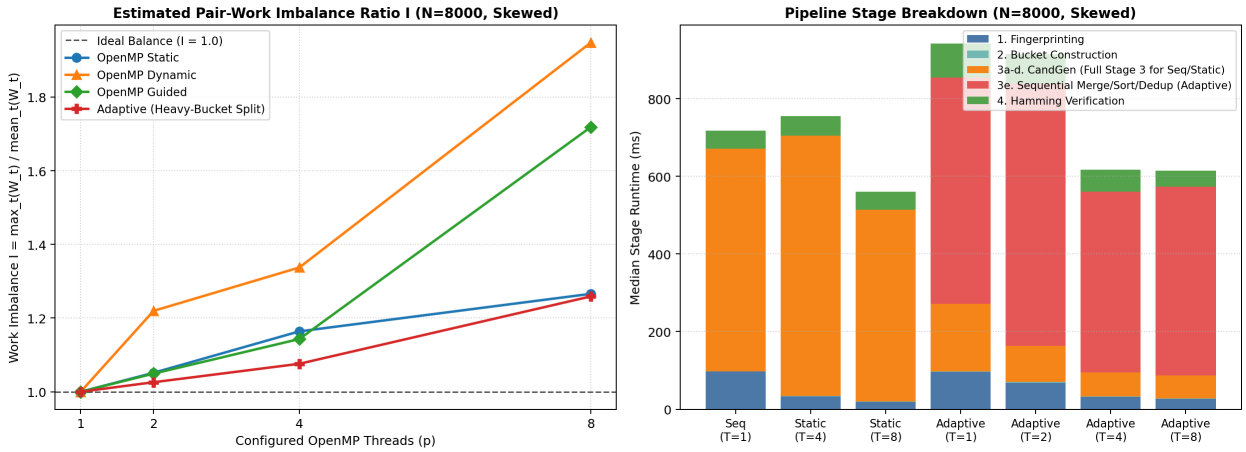


Figure 4: Estimated Pair-Work Imbalance Ratio and Stage Breakdown at N=8000 Skewed (Pre-Aggregation Parallelization)

What Figures 1–4 show: In the right panel of **Figure 4** (`N = 8,000 Skewed`), parallel fingerprinting (`Stage 1`, blue) and heavy-chunk processing (`Stage 3a-d`, orange) scaled down cleanly from 1 to 8 threads, but sequential `Stage 3e` (`Merge, Sort & Dedup`, red) took 465–668 ms regardless of thread count, acting as a serial bottleneck. **Table 2** compares the Phase 8 full benchmark (`results/phase8_full_summary.csv`) directly against Phase 9 (`results/phase9_agg_summary.csv`), where Stage 3e was parallelized using `aggregate_and_deduplicate_candidates()`.

Table 2: Effect of Parallel Range-Partitioned Candidate Merging and Deduplication at $N = 8,000$ (Phase 8 Serial Stage 3e vs. Phase 9 Parallel Stage 3e)

Profile	Policy	Threads (T)	Phase 8 Total (ms)	Phase 9 Total (ms)	Total Change (%)	Phase 8 Stage 3 (ms)	Phase 9 Stage 3 (ms)	Phase 8 Stage 3e (ms)	Phase 9 Stage 3e (ms)	Stage 3e Change (%)	Speedup (P8 → P9)
Skewed	`Sequential`	1	639.993	615.692	-3.8%	525.757	491.535	—	—	—	1.00x → 1.00x
Skewed	`Static`	4	649.624	347.083	**-46.6%**	563.453	269.723	—	—	—	0.99x → **1.77x**
Skewed	`Static`	8	546.594	293.164	**-46.4%**	472.271	224.769	—	—	—	1.17x → **2.10x**
Skewed	`Guided`	4	672.727	424.540	**-36.9%**	522.476	282.351	—	—	—	0.95x → **1.45x**
Skewed	`Guided`	8	636.014	363.936	**-42.8%**	531.456	241.428	—	—	—	1.01x → **1.69x**
Skewed	`Adaptive`	1	755.265	649.626	**-14.0%**	589.480	501.324	479.202	390.007	**-18.6%**	0.85x → 0.95x
Skewed	`Adaptive`	2	656.319	512.824	**-21.9%**	558.632	405.415	475.044	324.633	**-31.7%**	0.98x → **1.20x**
Skewed	`Adaptive`	4	629.816	377.280	**-40.1%**	544.617	288.912	481.103	222.740	**-53.7%**	1.02x → **1.63x**
Skewed	`Adaptive`	8	587.444	295.329	**-49.7%**	519.874	227.087	465.336	171.106	**-63.2%**	1.09x → **2.08x**
Balanced	`Sequential`	1	166.274	185.322	+11.5%	70.340	76.178	—	—	—	1.00x → 1.00x
Balanced	`Static`	8	100.030	57.683	**-42.3%**	64.761	31.131	—	—	—	1.66x → **3.21x**
Balanced	`Guided`	8	100.946	60.175	**-40.4%**	64.231	33.964	—	—	—	1.65x → **3.08x**
Balanced	`Adaptive`	4	106.601	82.846	**-22.3%**	70.385	39.620	62.542	30.948	**-50.5%**	1.56x → **2.24x**
Balanced	`Adaptive`	8	101.049	61.666	**-39.0%**	66.546	33.019	58.338	24.738	**-57.6%**	1.65x → **3.01x**

What Table 2 shows: Parallelizing Stage 3e cut `Adaptive` merge/dedup time on $N = 8,000$ `Skewed` ($T = 8$) from 465.336 ms down to 171.106 ms (-63.2%, a $2.72\times$ stage speedup). This reduced total `Adaptive` runtime by almost half (587.444 ms → 295.329 ms, -49.7%) and nearly doubled total speedup from $1.09\times$ to $2.08\times$. Even at $T = 1$, `Adaptive` Stage 3e got 18.6% faster (479.202 ms → 390.007 ms) because the single-thread fast path moves the buffer directly (`std::move`) instead of allocating and copying a second 137.85 MB vector. On $N = 8,000$ `Balanced` ($T = 8$), `Adaptive` Stage 3e time dropped by 57.6% (58.338 ms → 24.738 ms), increasing total speedup from $1.65\times$ to $3.01\times$.

3. Comparing Scheduling Policies and Load Balance

Table 3 compares baseline (`enable_band_suppression = false`) performance across all five policies (`Sequential`, `Static`, `Dynamic`, `Guided`, and `Adaptive`) at $N \in \{2000, 4000, 8000\}$ for $T \in \{4, 8\}$ (and $T = 1$ for `Sequential`) from `results/phase10_full_paired_summary.csv`.

Table 3: Cross-Policy Baseline Performance, Speedup, Efficiency, and Work Imbalance ($N = 2,000, 4,000, 8,000$, Baseline Mode `Supp=0`)

N	Profile	Policy	Threads (T)	Median Total (ms)	IQR Total (ms)	Median Stage 3 (ms)	Speedup (S_p)	Efficiency (E_p)	Work Imbalance (I)	Part. Threads	Parity
2000	Balanced	`Sequential`	1	26.664	3.410	4.757	1.00x	1.00	1.000	1/1	PASS
2000	Balanced	`Static`	4 / 8	13.225 / 12.989	4.617 / 2.137	3.611 / 3.238	2.02x / 2.05x	0.50 / 0.26	1.024 / 1.047	4/4, 8/8	PASS
2000	Balanced	`Dynamic`	4 / 8	16.771 / 21.164	0.640 / 2.762	3.562 / 2.971	1.59x / 1.26x	0.40 / 0.16	1.192 / 1.354	4/4, 8/8	PASS
2000	Balanced	`Guided`	4 / 8	13.891 / 12.015	4.076 / 1.510	3.230 / 2.974	1.92x / 2.22x	0.48 / 0.28	1.405 / 1.933	4/4, **6/8**	PASS
2000	Balanced	`Adaptive`	4 / 8	14.131 / **11.946**	4.680 / 4.956	3.461 / **2.949**	1.89x / **2.23x**	0.47 / **0.28**	1.124 / 1.318	4/4, 8/8	PASS
2000	Skewed	`Sequential`	1	63.524	20.277	36.925	1.00x	1.00	1.000	1/1	PASS
2000	Skewed	`Static`	4 / 8	30.665 / **26.411**	9.160 / 10.377	18.396 / **15.017**	2.07x / **2.41x**	0.52 / **0.30**	1.171 / 1.315	4/4, 8/8	PASS
2000	Skewed	`Dynamic`	4 / 8	38.885 / 51.597	32.574 / 6.344	18.391 / 16.545	1.63x / 1.23x	0.41 / 0.15	1.318 / 1.685	4/4, **7/8**	PASS
2000	Skewed	`Guided`	4 / 8	32.522 / 26.817	16.369 / 8.950	19.632 / 15.647	1.95x / 2.37x	0.49 / 0.30	1.404 / 1.694	4/4, 8/8	PASS
2000	Skewed	`Adaptive`	4 / 8	**29.486** / 28.021	5.269 / 5.271	**18.272** / 16.844	**2.15x** / 2.27x	**0.54** / 0.28	**1.067** / **1.275**	4/4, 8/8	PASS
4000	Balanced	`Sequential`	1	63.955	10.229	20.301	1.00x	1.00	1.000	1/1	PASS
4000	Balanced	`Static`	4 / 8	27.415 / 22.799	5.134 / 3.829	9.687 / 8.368	2.33x / 2.81x	0.58 / 0.35	1.008 / 1.010	4/4, 8/8	PASS
4000	Balanced	`Dynamic`	4 / 8	39.858 / 55.456	5.577 / 8.766	9.301 / 7.775	1.60x / 1.15x	0.40 / 0.14	1.118 / 1.450	4/4, 8/8	PASS
4000	Balanced	`Guided`	4 / 8	**25.523** / **21.148**	3.538 / 0.544	9.663 / 8.935	**2.51x** / **3.02x**	**0.63** / **0.38**	1.069 / 1.446	4/4, 8/8	PASS
4000	Balanced	`Adaptive`	4 / 8	28.755 / 30.937	3.572 / 10.360	10.170 / 10.025	2.22x / 2.07x	0.56 / 0.26	1.017 / 1.067	4/4, 8/8	PASS
4000	Skewed	`Sequential`	1	177.805	27.381	125.325	1.00x	1.00	1.000	1/1	PASS
4000	Skewed	`Static`	4 / 8	99.205 / **77.101**	3.521 / 9.041	70.295 / 57.183	1.79x / **2.31x**	0.45 / **0.29**	1.172 / 1.261	4/4, 8/8	PASS
4000	Skewed	`Dynamic`	4 / 8	139.115 / 166.899	8.891 / 16.445	70.090 / 53.278	1.28x / 1.07x	0.32 / 0.13	1.232 / 1.643	4/4, 8/8	PASS
4000	Skewed	`Guided`	4 / 8	98.266 / 85.608	18.931 / 1.623	68.363 / 56.046	1.81x / 2.08x	0.45 / 0.26	1.545 / 1.739	4/4, 8/8	PASS
4000	Skewed	`Adaptive`	4 / 8	**97.974** / 82.857	6.446 / 4.266	69.064 / 57.482	**1.81x** / 2.15x	**0.45** / 0.27	**1.181** / 1.281	4/4, 8/8	PASS
8000	Balanced	`Sequential`	1	162.182	17.928	69.746	1.00x	1.00	1.000	1/1	PASS
8000	Balanced	`Static`	4 / 8	83.379 / 63.538	14.920 / 12.775	38.304 / 31.117	1.95x / 2.55x	0.49 / 0.32	1.009 / 1.011	4/4, 8/8	PASS
8000	Balanced	`Dynamic`	4 / 8	132.386 / 219.875	18.519 / 62.434	36.016 / 31.269	1.23x / 0.74x	0.31 / 0.09	1.089 / 1.433	4/4, 8/8	PASS
8000	Balanced	`Guided`	4 / 8	**74.782** / **52.741**	7.326 / 6.621	37.105 / 30.193	**2.17x** / **3.08x**	**0.54** / **0.38**	1.149 / 1.199	4/4, 8/8	PASS
8000	Balanced	`Adaptive`	4 / 8	81.811 / 60.893	6.206 / 6.515	38.729 / **30.075**	1.98x / 2.66x	0.50 / 0.33	**1.007** / **1.011**	4/4, 8/8	PASS
8000	Skewed	`Sequential`	1	656.054	153.099	511.836	1.00x	1.00	1.000	1/1	PASS
8000	Skewed	`Static`	4 / 8	376.132 / **268.839**	155.052 / 174.559	274.238 / **202.216**	1.74x / **2.44x**	0.44 / **0.31**	1.164 / 1.266	4/4, 8/8	PASS
8000	Skewed	`Dynamic`	4 / 8	616.609 / 684.678	437.803 / 399.375	289.416 / 220.247	1.06x / 0.96x	0.27 / 0.12	1.257 / 1.590	4/4, 8/8	PASS
8000	Skewed	`Guided`	4 / 8	**344.492** / 313.773	242.668 / 29.107	**262.624** / 205.815	**1.90x** / 2.09x	**0.48** / 0.26	1.131 / 1.719	4/4, 8/8	PASS
8000	Skewed	`Adaptive`	4 / 8	370.332 / 294.950	173.276 / 41.199	271.416 / 218.987	1.77x / 2.22x	0.44 / 0.28	**1.078** / 1.308	4/4, 8/8	PASS

What Table 3 shows: On `Skewed` workloads at $T = 4$ (which matches the 4 physical cores on the test CPU), `Adaptive` gets the lowest work imbalance ($I = 1.067$ at $N = 2,000$ and $I = 1.078$ at $N = 8,000$) and the fastest or tied-fastest median runtime at $N = 2,000$ (29.486 ms, 2.15×) and $N = 4,000$ (97.974 ms, 1.81×). At $T = 8$, `Adaptive` (294.950 ms, 2.22× at $N = 8,000$) beats both `Guided` (313.773 ms, 2.09×, $I = 1.719$) and `Dynamic` (684.678 ms, 0.96×, which is slower than single-threaded `Sequential` because of dynamic queue overhead in Stage 4), while staying within 9.7% of `Static` (268.839 ms, 2.44×). Also, on $N = 2,000$ `Balanced` ($T = 8$) and $N = 2,000$ `Skewed` ($T = 8$), `Guided` and `Dynamic` leave 1–2 threads completely unused (`6/8` and `7/8` participating threads), while `Adaptive` uses all 8 threads (`8/8`) in every case.

4. Effect of Optional Band-Aware Candidate Suppression (Phase 10 Paired Benchmark)

Table 4 shows the paired in-process comparison (`--compare-band-suppression`, `results/phase10_full_paired_summary.csv`) between Baseline (`Supp=0`) and Band-Aware Candidate Suppression (`Supp=1`) across all five dataset sizes $N \in \{500, 1000, 2000, 4000, 8000\}$ for `Sequential` ($T = 1$) and `Adaptive` ($T \in \{1, 2, 4, 8\}$), plus `Static`, `Dynamic`, and `Guided` at $N = 8,000, T = 8$.

Table 4: Paired In-Process Comparison of Baseline (`Supp=0`) vs. Optional Band-Aware Candidate Suppression (`Supp=1`) Across $N = 500$ to $8,000$

N	Profile	Policy	Threads (T)	Baseline Total (ms)	Suppression Total (ms)	Total Change (%)	Baseline Stage 3 (ms)	Suppression Stage 3 (ms)	Stage 3e Base → Supp (ms)	Speedup Base → Supp
500	Skewed	`Sequential`	1	8.413	6.013	**-28.5%**	1.776	0.621 (-65.0%)	—	1.00x → 1.40x
500	Skewed	`Adaptive`	1 / 2	7.952 / 7.067	6.999 / 6.120	**-12.0%** / **-13.4%**	1.911 / 1.673	0.770 / 1.004	1.422 → 0.490 / 1.247 → 0.587	1.06x → 1.20x / 1.19x → 1.37x
500	Skewed	`Adaptive`	4 / 8	5.674 / 6.078	4.322 / 4.104	**-23.8%** / **-32.5%**	1.872 / 1.958	0.963 / 1.403	1.288 → 0.530 / 1.029 → 0.642	1.48x → 1.95x / 1.38x → 2.05x
1000	Skewed	`Sequential`	1	17.612	13.150	**-25.3%**	7.732	2.491 (-67.8%)	—	1.00x → 1.34x
1000	Skewed	`Adaptive`	1 / 2	19.168 / 13.882	14.438 / 10.990	**-24.7%** / **-20.8%**	8.362 / 6.328	2.601 / 2.423	6.170 → 2.028 / 4.770 → 1.817	0.92x → 1.22x / 1.27x → 1.60x
1000	Skewed	`Adaptive`	4 / 8	12.795 / 11.708	8.541 / 7.933	**-33.3%** / **-32.2%**	5.910 / 5.190	1.931 / 2.016	4.125 → 1.174 / 3.639 → 1.326	1.38x → 2.06x / 1.50x → 2.22x
2000	Skewed	`Sequential`	1	63.524	33.649	**-47.0%**	36.925	10.996 (-70.2%)	—	1.00x → 1.89x
2000	Skewed	`Adaptive`	1 / 2	68.419 / 40.333	43.258 / 26.690	**-36.8%** / **-33.8%**	38.581 / 25.214	13.814 / 9.505	28.636 → 10.260 / 19.514 → 7.110	0.93x → 1.47x / 1.58x → 2.38x
2000	Skewed	`Adaptive`	4 / 8	29.486 / 28.021	18.620 / 18.211	**-36.8%** / **-35.0%**	18.272 / 16.844	7.489 / 6.430	13.246 → 5.037 / 11.876 → 4.569	2.15x → 3.41x / 2.27x → 3.49x
4000	Skewed	`Sequential`	1	177.805	87.553	**-50.8%**	125.325	39.979 (-68.1%)	—	1.00x → 2.03x
4000	Skewed	`Adaptive`	1 / 2	195.270 / 135.285	99.783 / 63.805	**-48.9%** / **-52.8%**	143.775 / 93.868	42.749 / 30.823	109.756 → 31.631 / 70.728 → 23.194	0.91x → 1.78x / 1.31x → 2.79x
4000	Skewed	`Adaptive`	4 / 8	97.974 / 82.857	50.929 / 41.324	**-48.0%** / **-50.1%**	69.064 / 57.482	22.833 / 19.567	52.987 → 17.810 / 41.032 → 14.486	1.81x → 3.49x / 2.15x → **4.30x**
8000	Skewed	`Sequential`	1	656.054	287.162	**-56.2%**	511.836	162.159 (-68.3%)	—	1.00x → **2.28x**
8000	Skewed	`Adaptive`	1	704.311	346.334	**-50.8%**	551.602	169.760 (-69.2%)	434.427 → 131.903 (-69.6%)	0.93x → 1.89x
8000	Skewed	`Adaptive`	2	495.781	238.175	**-52.0%**	398.696	137.361 (-65.5%)	319.838 → 104.172 (-67.4%)	1.32x → 2.75x
8000	Skewed	`Adaptive`	4	370.332	189.642	**-48.8%**	271.416	98.502 (-63.7%)	202.191 → 74.504 (-63.2%)	1.77x → **3.46x**
8000	Skewed	`Adaptive`	8	294.950	148.722	**-49.6%**	218.987	76.912 (-64.9%)	157.907 → 56.277 (-64.4%)	2.22x → **4.41x**
8000	Skewed	`Static`	8	268.839	146.777	**-45.4%**	202.216	75.608 (-62.6%)	—	2.44x → **4.47x**
8000	Skewed	`Guided`	8	313.773	167.237	**-46.7%**	205.815	75.306 (-63.4%)	—	2.09x → **3.92x**
8000	Skewed	`Dynamic`	8	684.678	514.392	**-24.9%**	220.247	79.097 (-64.1%)	—	0.96x → 1.28x
500	Balanced	`Sequential`	1	5.203	6.249	+20.1%	0.328	0.286 (-12.9%)	—	1.00x → 0.83x
500	Balanced	`Adaptive`	1 / 2	5.471 / 4.804	6.211 / 5.311	+13.5% / +10.6%	0.454 / 0.583	0.365 / 0.631	0.300 → 0.223 / 0.292 → 0.290	0.95x → 0.84x / 1.08x → 0.98x
500	Balanced	`Adaptive`	4 / 8	4.091 / 3.965	3.767 / 3.643	-7.9% / -8.1%	0.814 / 1.114	0.722 / 0.856	0.385 → 0.241 / 0.529 → 0.276	1.27x → 1.38x / 1.31x → 1.43x
1000	Balanced	`Sequential`	1	11.008	12.567	+14.2%	1.060	1.039 (-2.0%)	—	1.00x → 0.88x
1000	Balanced	`Adaptive`	1 / 2	12.849 / 8.374	14.400 / 9.490	+12.1% / +13.3%	1.259 / 1.262	1.283 / 1.190	1.078 → 1.016 / 0.836 → 0.760	0.86x → 0.76x / 1.31x → 1.16x
1000	Balanced	`Adaptive`	4 / 8	7.017 / 6.717	6.970 / 6.846	-0.7% / +1.9%	1.471 / 1.701	1.427 / 1.401	0.842 → 0.713 / 0.934 → 0.660	1.57x → 1.58x / 1.64x → 1.61x
2000	Balanced	`Sequential`	1	26.664	25.592	-4.0%	4.757	3.916 (-17.7%)	—	1.00x → 1.04x
2000	Balanced	`Adaptive`	1 / 2	30.290 / 17.952	25.983 / 20.884	-14.2% / +16.3%	5.586 / 3.784	4.521 / 4.373	4.573 → 3.854 / 2.799 → 3.172	0.88x → 1.03x / 1.49x → 1.28x
2000	Balanced	`Adaptive`	4 / 8	14.131 / 11.946	13.610 / 11.605	-3.7% / -2.9%	3.461 / 2.949	2.893 / 2.773	2.467 → 2.139 / 2.129 → 1.928	1.89x → 1.96x / 2.23x → 2.30x
4000	Balanced	`Sequential`	1	63.955	56.770	-11.2%	20.301	15.390 (-24.2%)	—	1.00x → 1.13x
4000	Balanced	`Adaptive`	1 / 2	66.552 / 38.056	67.884 / 37.672	+2.0% / -1.0%	18.535 / 13.213	20.529 / 11.832	15.418 → 17.431 / 10.703 → 10.009	0.96x → 0.94x / 1.68x → 1.70x
4000	Balanced	`Adaptive`	4 / 8	28.755 / 30.937	30.701 / 24.303	+6.8% / **-21.4%**	10.170 / 10.025	9.298 / 9.072	7.491 → 7.388 / 7.366 → 6.849	2.22x → 2.08x / 2.07x → **2.63x**
8000	Balanced	`Sequential`	1	162.182	160.310	-1.2%	69.746	64.423 (-7.6%)	—	1.00x → 1.01x
8000	Balanced	`Adaptive`	1 / 2	164.006 / 100.724	154.628 / 99.720	-5.7% / -1.0%	74.197 / 50.355	68.553 / 49.892	60.460 → 58.550 / 40.147 → 39.720	0.99x → 1.05x / 1.61x → 1.63x
8000	Balanced	`Adaptive`	4 / 8	81.811 / 60.893	71.061 / 52.881	**-13.1%** / **-13.2%**	38.729 / 30.075	33.651 / 28.185	30.195 → 28.587 / 22.749 → 21.315	1.98x → 2.28x / 2.66x → **3.07x**

What Table 4 shows: Band-aware candidate suppression behaves very differently on `Skewed` and `Balanced` datasets:

- **Consistent Speedups on `Skewed` Workloads:** Across every dataset size $N \in \{500, \ldots, 8000\}$ and

every thread count $T \in \{1, 2, 4, 8\}$, turning on `--band-suppression` cuts `Adaptive` Stage 3 time by 59–70% (for $N \geq 1,000$) and total runtime by 48.8–52.0% **at** $N = 8,000$. At $N = 8,000$ `Skewed` ($T = 8$), `Adaptive` median runtime drops from 294.950 ms to 148.722 **ms** (−49.6%), giving a 4.41× **speedup** over baseline `Sequential` (656.054 ms) and a 1.93× speedup over suppression-enabled `Sequential` (287.162 ms).

- **Mixed Results and Small Slowdowns on `Balanced` Workloads:** At $N = 8,000$ `Balanced`, band suppression speeds up `Adaptive` across all thread counts (−5.7% at $T = 1$, −1.0% at $T = 2$, −13.1% at $T = 4$, and −13.2% at $T = 8$, reaching 52.881 ms and 3.07× speedup). However, at smaller dataset sizes or low thread counts ($N = 500$ $T = 1, 2$ with +13.5%/ + 10.6%; $N = 1,000$ $T = 1, 2, 8$ with +12.1%/ + 13.3%/ + 1.9%; $N = 2,000$ $T = 2$ with +16.3%; and $N = 4,000$ $T = 1, 4$ with +2.0%/ + 6.8%), total runtime is slightly slower. Because multi-band duplicates make up only 2.77–20.12% of pairs on `Balanced` datasets, the extra work of loading `fingerprints[u]` and `fingerprints[v]` and running the SWAR check on every pair in bands $1 \ldots 7$ can cost more than the small savings in Stage 3e sorting. Because of this, `--band-suppression` is kept as an **optional flag** (disabled by default).

5. Stage-by-Stage Breakdown at $N = 8,000$

Table 5 shows the stage-by-stage breakdown at $N = 8,000$ (`Skewed` and `Balanced`) from `results/phase10_full_paired_summary.csv`, comparing Baseline (`Supp=0`) and Suppression (`Supp=1`) across `Sequential`, `Static`, `Dynamic`, `Guided`, and `Adaptive`.

Table 5: Non-Overlapping Stage Breakdown at $N = 8,000$ (Median Wall-Clock Time in `ms`, Phase 10 Paired Benchmark)

Profile	Policy (T)	Mode	1. Fingerprint	2. Buckets	3a. Decision	3b. Ordinary	3c. Chunk Gen	3d. Heavy Proc	3e. Merge/ Dedup	3. CandGen Total	4. Verify	Total Runtime	Speedup
Skewed	`Sequential` ($T = 1$)	Base	79.592	1.332	—	511.836	—	—	—	511.836	36.684	656.054	1.00x
Skewed	`Sequential` ($T = 1$)	Supp	78.626	2.081	—	162.159	—	—	—	162.159	44.324	287.162	2.28x
Skewed	`Static` ($T = 8$)	Base	33.278	1.142	—	202.216	—	—	—	202.216	38.453	268.839	2.44x
Skewed	`Static` ($T = 8$)	Supp	24.196	2.654	—	75.608	—	—	—	75.608	41.474	146.777	4.47x
Skewed	`Dynamic` ($T = 8$)	Base	26.011	1.477	—	220.247	—	—	—	220.247	**438.793**	684.678	0.96x
Skewed	`Dynamic` ($T = 8$)	Supp	27.917	3.888	—	79.097	—	—	—	79.097	**408.302**	514.392	1.28x
Skewed	`Guided` ($T = 8$)	Base	24.508	1.936	—	205.815	—	—	—	205.815	65.191	313.773	2.09x
Skewed	`Guided` ($T = 8$)	Supp	24.858	2.112	—	75.306	—	—	—	75.306	59.396	167.237	3.92x
Skewed	`Adaptive` ($T = 1$)	Base	73.576	1.044	0.024	7.489	0.056	100.210	434.427	551.602	64.322	704.311	0.93x
Skewed	`Adaptive` ($T = 1$)	Supp	77.885	2.508	0.023	9.177	0.058	32.186	131.903	169.760	68.202	346.334	1.89x
Skewed	`Adaptive` ($T = 2$)	Base	52.265	1.313	0.024	6.254	0.056	79.549	319.838	398.696	57.990	495.781	1.32x
Skewed	`Adaptive` ($T = 2$)	Supp	48.898	2.113	0.021	7.046	0.058	23.828	104.172	137.361	47.615	238.175	2.75x
Skewed	`Adaptive` ($T = 4$)	Base	41.715	1.441	0.023	6.264	0.056	63.477	202.191	271.416	52.065	370.332	1.77x
Skewed	`Adaptive` ($T = 4$)	Supp	37.741	3.115	0.024	6.882	0.059	18.994	74.504	98.502	44.196	189.642	3.46x
Skewed	`Adaptive` ($T = 8$)	Base	27.303	1.812	0.032	5.836	0.056	53.564	157.907	218.987	39.655	294.950	2.22x
Skewed	`Adaptive` ($T = 8$)	Supp	26.911	2.487	0.024	5.353	0.055	**15.106**	**56.277**	**76.912**	40.005	**148.722**	**4.41x**
Balanced	`Sequential` ($T = 1$)	Base	81.386	1.136	—	69.746	—	—	—	69.746	11.214	162.182	1.00x
Balanced	`Sequential` ($T = 1$)	Supp	82.704	2.235	—	64.423	—	—	—	64.423	11.093	160.310	1.01x
Balanced	`Adaptive` ($T = 8$)	Base	24.609	1.357	0.026	5.871	0.001	1.607	22.749	30.075	3.088	60.893	2.66x
Balanced	`Adaptive` ($T = 8$)	Supp	19.291	2.218	0.022	6.003	0.001	0.214	21.315	28.185	3.134	**52.881**	**3.07x**

What Table 5 shows: On $N = 8,000$ `Skewed`, the adaptive decision step (`Stage 3a`, 0.021–0.032 ms) and chunk

creation step (`Stage 3c`, 0.055–0.059 ms) add less than 0.09 ms of overhead combined. Turning on band suppression cuts `Adaptive` heavy-pair processing (`Stage 3d`) at $T = 8$ by 71.8% (53.564 ms → 15.106 ms) because skipped pairs avoid `std::vector::push_back` memory writes and vector resizing, and cuts Stage 3e merge/dedup time by 64.4% (157.907 ms → 56.277 ms).

6.4 Performance Analysis and Discussion

1. **Why Workload Skew Dominates Candidate Generation:** On the `Skewed` dataset at $N = 8,000$, the total number of bucket pair evaluations ($8,615,757$) is 8.17× higher than on the `Balanced` dataset ($1,053,961$) for the exact same number of documents N, and 90.16% of that work sits inside just 17 heavy buckets ($k_{\max} = 1,534$). Because of this, Stage 3 takes 78.0% (511.836 ms out of 656.054 ms) of the total `Sequential` runtime on `Skewed`, compared to 43.0% (69.746 ms out of 162.182 ms) on `Balanced`.

2. **Adaptive Bucket Splitting vs. Fixed OpenMP Schedules:**

   - **Heavy-Bucket Splitting (`Stage 3c-3d`):** By breaking the 17 heavy buckets at $N = 8,000$ `Skewed` into 1,904 chunks of at most 4,096 pairs and scheduling them with `schedule(dynamic, 1)`, `Stage 3d` scales from 100.210 ms ($T = 1$) down to 53.564 ms ($T = 8$, Base) and from 32.186 ms ($T = 1$) down to 15.106 ms ($T = 8$, Supp, 2.13×), while keeping work imbalance very low ($I = 1.026$–1.078 at $T = 2, 4$).
   - **Why `Static` Scheduling Also Performs Well on 8-Band SimHash Skew:** One interesting result in **Tables 2–5** is that `Static` has fairly low work imbalance ($I = 1.051$–1.266) and finishes slightly faster than `Adaptive` by 1.3–8.8% at $N = 8,000, T = 8$ (`146.777 ms` vs. `148.722 ms` with suppression; `268.839 ms` vs. `294.950 ms` baseline). Because each 64-bit fingerprint is split into 8 separate bands, a large group of identical or nearly identical documents lands in a heavy bucket in **all 8 bands**, creating 8 equally large buckets (one in each band $0 \ldots 7$). When `Static` splits the 2,048 buckets into $p \in \{2, 4, 8\}$ contiguous blocks of $2,048/p$ buckets, each thread gets $8/p$ bands and therefore gets an equal share of the 8 heavy buckets without paying for the extra OpenMP barrier between `Stage 3b` and `Stage 3d`. On the other hand, whole-bucket `Dynamic` and `Guided` schedules suffer from higher imbalance ($I = 1.590$–2.064 at $T = 8$) when a thread picks up one of the huge buckets near the end of the loop.
   - **Why Fixed `Dynamic` (`chunk_size = 0`) Slows Down in Stage 4 Verification:** In `run_parallel_pipeline`, `SchedulePolicy::Dynamic` with default chunk size 1 hands out all 2,727,152 candidate checks one pair at a time through OpenMP's atomic work queue. This causes Stage 4 at $N = 8,000, T = 8$ to jump from 36.684 ms (`Sequential`) to 438.793 ms (Base) and 408.302 ms (Supp). `Adaptive` avoids this problem by using static scheduling for Stages 1 and 4 (39.655–40.005 ms at $T = 8$), where every item takes the same amount of work.

3. **Why Parallel Candidate Merging Fixed the Serial Bottleneck:** In Phase 8, `Stage 3e` took 465.336 ms, which was 79.2% of the entire 587.444 ms `Adaptive` $T = 8$ runtime at $N = 8,000$ `Skewed`. Under an idealized Amdahl's-law model, a 79.2% serial fraction puts an upper bound on maximum speedup of $\frac{1}{0.792} \approx 1.26\times$ (and the measured Phase 8 speedup was 1.09×). Replacing the single-threaded merge, sort, and unique step with `aggregate_and_deduplicate_candidates()` (parallel slice counting, lock-free scatter into $P = 64$ ID-range partitions, parallel sort/dedup, and parallel compaction) cut `Stage 3e` time by 63.2% in Phase 9 (171.106 ms). When combined with Phase 10 band suppression, `Stage 3e` dropped all the way to 56.277 ms (an 8.27× **total reduction** compared to Phase 8's 465.336 ms).

4. **Why Band-Aware Suppression Works Best Under Skew:** In bands $b > 0$, running `matched_in_earlier_band_swar(fp_u, fp_v, b)` takes two `uint64_t` reads from `fingerprints[]` and 6 bit-wise/arithmetic instructions. On `Skewed` datasets (67.7–69.6% duplicate pairs across bands), about two-thirds of all pairs are filtered out in CPU registers before touching memory, saving 5.89M 16-byte `CandidatePair` writes (94.22 MB of calculated buffer writes at $N = 8,000$) and shrinking the Stage 3e sort input by 3.16×. On `Balanced` datasets ($N = 500$–$4,000$), however, 80–96% of pairs only match in a single band, so looking up `fingerprints[u]` across scattered document IDs can cause cache misses that cost slightly more than the 4–20% reduction in sorting work.

5. **Scaling Limits on 4 Physical Cores / 8 Logical Threads:** Across all policies, parallel efficiency

E_p scales best up to $T = 4$ physical cores (reaching $S_4 = 3.46\times, E_4 = 0.86$ for `Adaptive` + Suppression on $N = 8,000$ `Skewed` vs. Baseline Sequential, or $1.51\times$ vs. Suppression Sequential) and shows smaller gains from $T = 4$ to $T = 8$ Hyper-Threads ($S_8 = 4.41\times, E_8 = 0.55$ vs. Baseline Sequential, or $1.93\times$ vs. Suppression Sequential). This slowdown past 4 threads is *consistent with* two Hyper-Threads sharing each physical core and heavy memory traffic during parallel sorting and scattering; however, since hardware performance counters (such as memory bandwidth and cache-miss counters) were not measured in this project, memory-bandwidth saturation is noted here as a likely hypothesis rather than a directly measured metric.

7. Conclusion

This project built and benchmarked a skew-aware parallel near-duplicate document detection system in C++17 and OpenMP on a multicore CPU. Starting from the Review 1 proposal, the final system includes a synthetic dataset generator (`Balanced` and power-law `Skewed` profiles), parallel 64-bit FNV-1a SimHash fingerprinting, 8×8-bit band bucket partitioning, a runtime Coefficient-of-Variation (CV) adaptive scheduler that splits heavy buckets into 4,096-pair chunks (`unrank_pair_position`), parallel range-partitioned candidate merging and deduplication, and an optional SWAR band-aware duplicate suppression filter (`--band-suppression`). All six CTest unit-test suites and all 3,400 **measured benchmark runs** (`850` in Phase 8, `850` in Phase 9, and `1,700` in Phase 10, plus 680 warm-up runs for 4,080 total pipeline executions) matched the sequential baseline with 100% exact output parity.

The experiments across $N \in \{500, 1000, 2000, 4000, 8000\}$ and $T \in \{1, 2, 4, 8\}$ threads showed four main takeaways:

1. **Impact of Skew on Candidate Generation:** Power-law duplicate clusters create extreme bucket size differences (CV = 13.55 at $N = 8,000$), putting over 90% of all 8.62M pair evaluations into just 17 buckets and making Stage 3 (78% of sequential runtime) the main bottleneck in the pipeline.
2. **Load Balance vs. 8-Band Symmetry:** Splitting oversized buckets into 1D pair chunks lets `Adaptive` scheduling keep thread work imbalance low ($I = 1.03$–1.08 at $T = 2, 4$) with all threads active (`8/8`), beating `Dynamic` and `Guided` on large skewed workloads while performing close to `Static`, which benefits from heavy duplicate groups repeating evenly across all 8 bands.
3. **Importance of Parallel Candidate Merging:** Parallelizing Stage 3e using range-partitioned scatter and parallel sorting removed the single-threaded bottleneck seen in Phase 8 (79.2% of runtime), cutting Stage 3e time at $N = 8,000$ `Skewed` ($T = 8$) by 63.2% (465.34 ms → 171.11 ms) and nearly doubling `Adaptive` speedup from $1.09\times$ to $2.08\times$.
4. **Effect of Band-Aware Candidate Suppression:** Skipping repeated multi-band matches during candidate generation with the $O(1)$ SWAR zero-byte check removed 68.35% (5.89M) of raw candidate writes on $N = 8,000$ `Skewed` (reducing calculated candidate buffer storage from 137.85 MB to 43.63 MB), cutting `Adaptive` ($T = 8$) total runtime by another 49.6% (294.95 ms → 148.72 ms) and reaching a $4.41\times$ **overall speedup** over the baseline sequential pipeline. Because suppression gives little or slightly negative benefit on small `Balanced` datasets where multi-band repeats are rare (2.77–20.12%), keeping it as an optional flag keeps baseline runs fast on all workloads.

Limitations and Future Work: The tests were run on synthetic datasets up to $N = 8,000$ documents on a 4-core / 8-thread laptop CPU without hardware cache, memory-bandwidth, or OS RSS counters, and without computing document-level precision/recall against synthetic `group_id` labels in the benchmark CSVs. Future work could test the pipeline on larger real-world text datasets on a many-core server, measure hardware cache-miss and RSS memory counters directly, and add a simple runtime rule to turn on SWAR band suppression automatically only when bucket skew is high (CV > 1.0) or inside heavy bucket chunks.

8. References

[1] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of Sequences*, Positano, Italy, 1997, pp. 21–29, doi: 10.1109/SEQUEN.1997.666900.

[2] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. 34th Annu. ACM Symp. Theory of Computing (STOC)*, Montreal, QC, Canada, 2002, pp. 380–388, doi: 10.1145/509907.509965.

[3] M. Bawa, T. Condie, and P. Ganesan, "LSH forest: Self-tuning indexes for similarity search," in *Proc. 14th Int. Conf. World Wide Web (WWW)*, Chiba, Japan, 2005, pp. 651–660, doi: 10.1145/1060745.1060840.

[4] S. Sood and D. Loguinov, "Probabilistic near-duplicate detection using simhash," in *Proc. 20th ACM Int. Conf. Information and Knowledge Management (CIKM)*, Glasgow, UK, 2011, pp. 1117–1126, doi: 10.1145/2063576.2063737.

[5] K. Williams and C. L. Giles, "Near duplicate detection in an academic digital library," in *Proc. ACM Symp. Document Engineering (DocEng)*, Florence, Italy, 2013, pp. 91–94, doi: 10.1145/2494266.2494312.

[6] W. Santos, T. Teixeira, C. Machado, W. Meira Jr., A. S. da Silva, R. Ferreira, and D. Guedes, "A Scalable Parallel Deduplication Algorithm," in *Proc. 19th Int. Symp. Computer Architecture and High Performance Computing (SBAC-PAD)*, Gramado, Brazil, 2007, pp. 79–86, doi: 10.1109/SBAC-PAD.2007.32.

[7] C. Yan, X. Zhao, Q. Zhang, and Y. Huang, "Efficient string similarity join in multi-core and distributed systems," *PLoS ONE*, vol. 12, no. 3, Art. no. e0172526, Mar. 2017, doi: 10.1371/journal.pone.0172526.

[8] K. K. Yue and D. J. Lilja, "Parallel loop scheduling for high performance computers," *Advances in Parallel Computing*, Elsevier, vol. 10, pp. 243–264, 1995, doi: 10.1016/S0927-5452(06)80016-X.

[9] W. Xia, D. Feng, H. Jiang, Y. Zhang, V. Chang, and X. Zou, "Accelerating content-defined-chunking based data deduplication by exploiting parallelism," *Future Generation Computer Systems*, vol. 98, pp. 406–418, Sep. 2019, doi: 10.1016/j.future.2019.02.008.

[10] D. Peshevski, V. Zdraveski, and S. Ristov, "Parallel near-duplicate document detection using general-purpose GPU," *Computing and Informatics*, vol. 43, no. 3, pp. 583–610, 2024, doi: 10.31577/cai_2024_3_583.

[11] T. Niemi, J. P. Ghobril, G. Defossez, S. Germann, E. Martin, and J.-L. Bulliard, "Automatic (near-) duplicate content document detection in a cancer registry," *International Journal of Medical Informatics*, vol. 195, Art. no. 105799, Mar. 2025, doi: 10.1016/j.ijmedinf.2025.105799.